

Professionalization Review of “Licitaciones Mercado Público” Automation System

This assessment is based on the provided code snapshot (Repomix bundle) containing the Python orchestration script (`run_pipeline.py`), stage implementations (`src/etapas/etapa0.py` ... `src/etapas/etapa5.py`), incremental subsystem (`src/sistema_incremental.py`, `src/reporte_incremental.py`, `src/utils/analizador_incremental.py`), and shared utilities (`src/utils/*`). The Excel artifacts (`PIVOT_MAESTRO.xlsx`, outputs) were not part of the snapshot, so Excel schema and business rules are inferred from code expectations and sheet names.

Architecture and design

Problems found

Orchestration is hard-wired and stage integration is “filesystem-coupled” instead of “artifact-coupled.”

`run_pipeline.py` imports each stage class directly and executes them via explicit `if N in etapas:` blocks (Pipeline orchestrator: `PipelineLicitaciones` in `run_pipeline.py`, around lines 20–200). As a result: - Adding a new stage requires editing the orchestrator (imports + control flow + summary printing). - Stages discover inputs by “latest file in a folder” globbing rather than receiving explicit artifacts from previous stages (e.g., `src/etapas/etapa3.py::_obtener_archivo_entrada` uses `FILTRADO_DIR.glob("Licitaciones_Filtradas_*.xlsx")`, around lines 87–95; `src/etapas/etapa4.py::_obtener_archivo` uses `ENRIQUECIDO_DIR.glob("Licitaciones_Enriquecidas_*.xlsx")`, around lines 115–121).

This couples stages to naming conventions and directory structure, and increases the risk of mixing artifacts across runs (especially if two executions overlap or an old file remains “latest”).

No explicit “Stage contract” (interface) and no shared pipeline context model.

Each stage returns a loosely-typed dict (`{'exito': True, 'stats': ...}`), but the keys differ between stages (e.g., Stage 3 returns `total_enriquecidas`, Stage 2 returns `total_filtradas`, etc.). There is no typed context carrying `run_id`, input artifact paths, outputs, and metadata across all stages.

Import-path manipulation instead of proper packaging.

Multiple modules inject paths via `sys.path.append(...)` (e.g., `src/etapas/etapa1.py` lines ~9–13; `src/etapas/etapa2.py` lines ~7–12; `src/etapas/etapa3.py` lines ~14–18; `src/etapas/etapa4.py` lines ~14–16; `src/sistema_incremental.py` lines ~4–9; `src/reporte_incremental.py` lines ~9–12). This is a common source of environment-specific breakage and makes “install & run” much harder than it needs to be, especially for new developers.

Configuration is a mix of constants, Excel-driven config, and in-code assumptions.

`src/utils/config.py` defines many defaults as class constants and also reads dynamic parameters from `PIVOT_MAESTRO.xlsx` (`Config.cargar_desde_pivot`, around lines 135–182). Stages use a mix of `Config` as a class vs `Config()` as an instance (e.g., `Etapas0Descarga` sets

`self.config = Config` while `EnriquecedorAPI` uses `config or Config()`. This inconsistency increases mental overhead and complicates dependency injection/testing.

Side effects at import time.

`src/utils/config.py` calls `Config.validar_estructura()` at import (end of file, around lines 205-206). Import-time directory creation and warnings complicate unit tests and any tooling that imports modules without intending to run the pipeline.

Impact

High. The current architecture makes the system harder to evolve safely, complicates reliable re-runs, and increases the cost of adding or replacing pipeline stages.

Solution proposed

Introduce an explicit stage contract + a pipeline context + artifact passing (not folder-globbing).

A minimal, high-leverage refactor is to define: - A `PipelineContext` (run metadata, config, artifact registry). - A `Stage` protocol/ABC with `name`, `run(context) -> context`, and optional `validate_inputs(context)`.

Example (illustrative):

```
from dataclasses import dataclass, field
from pathlib import Path
from typing import Any, Dict, Protocol

@dataclass
class PipelineContext:
    run_id: str
    config: Any
    artifacts: Dict[str, Path] = field(default_factory=dict)
    metrics: Dict[str, Any] = field(default_factory=dict)

class Stage(Protocol):
    name: str
    def run(self, ctx: PipelineContext) -> PipelineContext: ...
```

Then stages produce named artifacts (`ctx.artifacts["filtered_xlsx"] = path`) and downstream stages consume those explicit keys (with a fallback mechanism if needed). This eliminates accidental “wrong latest file” behavior and makes a stage replaceable without changing every other stage.

Move toward a plugin-like registry for stages.

Instead of hard-coded imports + branching, define a stage registry:

```
STAGES = {
    0: Etapa0DescargaStage,
    1: AuditorTaxonomiaStage,
```

```

2: FiltradorStage,
3: EnriquecedorStage,
4: ReporteStage,
5: IncrementalStage,
}

```

The orchestrator iterates through selected stage IDs, retrieving from registry. This makes “add a new stage” mostly additive (new file/class + registry entry), not invasive.

Consider adopting a workflow orchestrator when/if operations mature.

If you expect growth in stages, schedules, retries, observability, and resuming from partial failures, frameworks like Apache Airflow ¹ or Prefect ² are purpose-built for DAG execution, retries, state, and resuming. Prefect explicitly supports task retries and caching/persisted results, which maps well to “don’t repeat work” and “resume a failed step.” ³

Priority of implementation

Short term. Start with the stage contract + context + explicit artifacts (highest ROI refactor). Orchestrator frameworks are a longer-term decision unless ops requirements justify them immediately.

Robustness and error handling

Problems found

Network resiliency is inconsistent and not centralized.

- Stage 0 (`src/etapas/etapa0.py`) uses `requests.head` and `requests.get` with timeouts and limited exception handling, but no structured retries/backoff for transient failures (download execution around lines 84–175).

- Stage 3 (`src/etapas/etapa3.py::_consultar_api`) has some retry behavior and special rules for 429/500 (around lines 136–181), but the retry strategy is embedded in the stage rather than shared/reusable across HTTP calls. - Stage 4 calls two different external APIs (UTM + FX rate) and catches exceptions, but one call embeds a credential (see security section) and both calls have no caching/health fallback beyond “log warning, keep defaults.” (`src/etapas/etapa4.py::_obtener_utm/_obtener_usd` , around lines 104–113)

No robust “resume from mid-pipeline” or checkpointing within long steps.

Stage 3 enrichment processes rows sequentially and only writes the final enriched output at the end (`_enriquecer_licitaciones` builds an in-memory list and then creates a DataFrame). If the process fails at item 150/200, you lose all progress and must restart Stage 3. (`src/etapas/etapa3.py` , enrichment loop around lines 110–134)

Input/output validation is partial and not uniform.

There is some validation (e.g., `validar_archivo_excel` checks existence/sheets in `src/utils/file_ops.py` , around lines 19–60), but: - There is no standardized “DataFrame contract” between stages (required columns, types, constraints). - Many operations rely on best-effort column matching and silent fallbacks (e.g., Stage 4 returns 999 days when parsing fails; Stage 3 returns `_api_error` fields). These are pragmatic, but the pipeline does not enforce sanity checks (e.g., “filtered count must be > 0” or “enriched success rate must be above X%”) that would catch upstream schema drift early.

Error signaling is mixed between “return dict with exito” and “raise Exception.”

Some stages return `{'exito': False, ...}`; others log and re-raise; the orchestrator sometimes throws generic `Exception("Etapa N fallo")`. This reduces diagnosability and makes it harder to implement consistent retry/resume policies.

Impact

High. Daily production automation touching external dependencies (web download + API + Excel IO) will encounter transient failures. Without central retry policies, checkpointing, and sanity checks, reliability will degrade and manual intervention will increase.

Solution proposed

Centralize HTTP client behavior with retries, backoff, and timeouts.

Requests does not retry failed connections by default; you must configure retries explicitly (for example using `HTTPAdapter` + `urllib3 Retry`). ⁴

Example: a shared `http.py` utility:

```
from requests import Session
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry

def build_retrying_session() -> Session:
    s = Session()
    retries = Retry(
        total=5,
        connect=5,
        read=5,
        backoff_factor=0.5,
        status_forcelist=[429, 500, 502, 503, 504],
        allowed_methods={"HEAD", "GET"} # keep idempotent by default
    )
    adapter = HTTPAdapter(max_retries=retries, pool_connections=20,
pool_maxsize=20)
    s.mount("https://", adapter)
    s.mount("http://", adapter)
    return s
```

Then Stage 0/3/4 call through this shared session rather than embedding retry logic per stage. (This also enables consistent logging and metrics around requests.)

For finer-grained retry policies (e.g., retry only on specific exceptions, add jitter, cap total retry duration), consider a retry library like Tenacity wrappers such as `stamina`. ⁵

Add checkpointing to Stage 3 enrichment and “resume mode” to the pipeline.

A practical pattern: - Maintain a durable cache or checkpoint file keyed by `codigo` (e.g., SQLite, JSONL, Parquet). - On start, load checkpoint, skip codes already enriched. - Append each successful (or failed) API result to checkpoint immediately (or every N items). - At the end, materialize Excel.

This converts Stage 3 from “all-or-nothing” to “incrementally recoverable.”

Implement “pipeline run manifest” to enable restart from last successful stage.

Write `manifest.json` (or similar) per `run_id` at the start of the pipeline; after each stage, record: - status, start/end timestamps - input/output artifact paths - row counts, key metrics - error summary if failed

On startup with `--resume`, read manifest and skip completed stages if outputs are present and valid.

Standardize exception taxonomy + stage results.

Define custom exceptions like `InputValidationError`, `ExternalServiceError`, `ArtifactNotFoundError`, and ensure each stage: - validates inputs early - raises typed exceptions - returns a typed result or updates context consistently

Priority of implementation

Short term. Central HTTP retry/timeouts + checkpointing Stage 3 are among the highest reliability improvements per unit of work.

Code quality

Problems found

Inefficient and duplicated text normalization in filtering.

`src/utils/text_processing.py::contiene_palabras_clave` normalizes each keyword inside the loop for every row check (around lines 196–201), even though Stage 2 already normalizes filter values when loading filters (`src/etapas/etapa2.py::_cargar_filtros`, lines ~70–85). This is classic hidden duplication that increases CPU time significantly as volume grows.

Row-wise `DataFrame.apply` used multiple times for inclusion/exclusion/bypass.

Stage 2 filtering is implemented as Python-level row functions (`df.apply(cumple, axis=1)`), repeated for inclusion, exclusion, and bypass (`src/etapas/etapa2.py`, around lines 129–173). For 12k rows this might still be acceptable, but it won’t scale cleanly and is hard to optimize further without refactoring.

Scripts contain repeated “main()” boilerplate and `sys.path` hacks.

Multiple stage files implement `main()` and CLI execution patterns, each with custom prints and error handling, in addition to the orchestrator. This increases maintenance surface, and changes to execution behavior must be replicated across files.

Dependency declaration is incomplete.

`requirements.txt` lists only `pandas`, `openpyxl`, `pytest`, `python-dateutil`, but the code imports `requests` and `pytz` (e.g., `src/etapas/etapa0.py`, `src/etapas/etapa3.py`, `src/etapas/etapa4.py`). This can break fresh installs and violates the “explicit dependencies” principle.

6

Type hints are partial and inconsistent.

Some components use typing, but many functions accept/return loosely typed dicts or DataFrames without typed schemas. Wider use of type hints improves refactoring safety and enables static checking; this is the core motivation of Python’s type-hinting standard (PEP 484). 7

Testing is mostly a “system test script,” not unit/contract tests.

`test_sistema_completo.py` acts as an integration runner and prints results. It's valuable, but there are few isolated unit tests: - No tests for filter logic edge cases (keyword collisions, normalization). - No tests for API response parsing in `_procesar_respuesta_api` (Stage 3). - No tests for “report formatting invariants” (column presence, days calculation). Pytest fixtures help structure unit/integration tests cleanly. ⁸

Impact

Medium-High. The system works today but is becoming costly to modify safely. Hidden performance issues and missing dependency hygiene are immediate friction points.

Solution proposed

Pre-normalize keywords once and compile matchers.

Replace repeated normalization inside `contiene_palabras_clave` with: - keywords normalized once at load time - a compiled regex (or Aho-Corasick if keyword lists grow large) - vectorized `Series.str.contains`

Example (simple and fast enough for many teams):

```
import re

def compile_keywords(words: list[str]) -> re.Pattern | None:
    clean = [w.strip() for w in words if w and w.strip()]
    if not clean:
        return None
    escaped = [re.escape(w) for w in clean]
    return re.compile("|".join(escaped), re.IGNORECASE)

# usage:
# pattern = compile_keywords(filtros["incluir"]["nombre"])
# mask = df["Nombre Adquisición (norm)"].str.contains(pattern, na=False)
```

Refactor Stage 2 filtering to compute boolean masks instead of per-row apply.

Compute masks per column and combine (`|` / `&`) according to business logic. This yields large speedups with the same semantics.

Introduce packaging and remove sys.path mutations.

Adopt `pyproject.toml` for project metadata and tool configuration; it is the standard modern mechanism for Python packaging and tool settings. ⁹

This also enables clean entrypoints like `python -m licitaciones_pipeline run ...`.

Add type hints where they reduce risk the most.

Prioritize: - Stage result objects (replace dicts with dataclasses / TypedDict / Pydantic models) - Config model - API response parsing result model This aligns with PEP 484 goals (static analysis, refactoring safety). ⁷

Build a pragmatic test suite.

Minimum critical unit tests: - `contiene_palabras_clave` and normalization behavior (accents, NaN, casing). - Stage 2: inclusion/exclusion/bypass with small DataFrames. - Stage 3: `_procesar_respuesta_api` against representative API payloads (including missing nested nodes). - Stage 4: currency conversion + days-to-close edge cases. Use pytest fixtures to keep tests readable and scalable. ⁸

Priority of implementation

Quick win → short term. Dependency fixes + keyword pre-normalization + first unit tests provide immediate confidence and speed wins without massive restructuring.

Performance and scalability

Problems found

API enrichment is sequential and rate-limited via `time.sleep`, limiting throughput.

Stage 3 enriches records one by one with an unconditional delay (`time.sleep(self.delay_segundos)` in `_enriquecer_licitaciones`, around lines 110–127). This is easy and safe, but: - It does not persist partial progress (robustness issue). - It wastes time when some calls fail fast or are cached. - It cannot easily scale if “filtered relevant” grows beyond ~200/day or if enrichment expands.

Filtering does repeated Python-level loops.

As discussed, Stage 2 uses row-wise apply and per-keyword checks (Stage 2 + `contiene_palabras_clave`). This becomes a bottleneck as volume doubles or as filter lists expand.

Excel read/write patterns may become heavy if outputs grow or if more sheets are added.

Openpyxl and pandas Excel operations can be memory-intensive at scale; openpyxl provides read-only and write-only optimized modes for large files. ¹⁰

If the system ever expands to produce larger “analyst workbooks,” these patterns matter.

Impact

Medium. At the current “12k → ~200” scale, the system is likely acceptable, but there are clear bottlenecks if the filtered set or enrichment complexity grows.

Solution proposed

Stage 3: introduce durable caching + controlled concurrency.

Durable cache often beats concurrency for “daily batch enrichment”: - Cache results by `codigo` and only call API for new/changed codes. - If concurrency is desired, use a semaphore + rate limiter so you never exceed the portal's constraints. A workflow framework can also manage retries and caching/persistence patterns cleanly (e.g., Prefect retries/caching). ¹¹

Stage 2: vectorize matching and reduce per-row Python execution.

Move to compiled patterns and column-wise masks. This typically gives order-of-magnitude improvements in pandas-based pipelines.

Be deliberate about Excel scaling strategy.

If outputs expand: - use openpyxl optimized modes when possible (read-only for reading large xlsx; write-only for generating large outputs). ¹⁰ - or introduce intermediary formats (Parquet) for heavy processing and convert to Excel only at the end.

Priority of implementation

Short term. Durable caching + mask-based filtering provide the best cost/benefit now. Concurrency is optional and should be introduced only with rate-limit guarantees.

Observability and operations

Problems found

Logging is present but not structured for production diagnostics and alerting.

`ProjectLogger` writes to a timestamped file and stdout and prints a “RESUMEN DE EJECUCION” with counts per log level (`src/utils/logger.py`, around lines 125–156). This is helpful, but: - There’s no stable run identifier propagated across stages (so correlating multi-stage logs is harder than necessary). - Logs are plain text without structured fields (stage, run_id, artifact_path, record counts, API error rates), limiting queryability in log platforms. - No log rotation policy; log directories may grow without bound. Python’s logging cookbook documents `RotatingFileHandler` patterns. ¹²

No business metrics are emitted automatically.

For a daily system, you want durable metrics like: - licitaciones downloaded - licitaciones filtered - enrichment success rate - average API latency and error reasons - report generation time Prometheus client libraries and instrumentation best practices are documented and straightforward to adopt. ¹³

No automated anomaly alerts.

Examples that should trigger alerts: - Stage 2 filtered = 0 (likely schema drift or filter config issue) - API success rate suddenly drops (portal issue, key expired, changed schema) - Report output missing expected columns Right now, these conditions may only appear as warnings in a log file.

Logging approach is more “file-based application logging” than “event stream.”

For professional operations, treat logs as an event stream (stdout with aggregation), as described in the Twelve-Factor principle on logs. ¹⁴

Impact

High. Observability is what makes systems reliable at scale. Without metrics + alerting, operators learn about failures only when analysts complain.

Solution proposed

Add a run_id and structured log context across all stages.

At pipeline start, generate `run_id` and inject into all logs. Even without full JSON logging, prefix every log line with `run_id` + `stage`.

Introduce metrics instrumentation (lightweight).

A pragmatic approach: - Emit a compact `metrics.json` per run in `OUTPUT/LOGS`. - Optionally expose Prometheus metrics if you have a runtime that can scrape/push.

Example metric set (conceptual): - `pipeline_stage_duration_seconds{stage="etapa3"}` - `licitaciones_filtered_total` - `api_calls_total`, `api_errors_total{status="500"}`, `api_success_rate`

Prometheus guidance emphasizes counters/gauges/histograms and exception counting patterns. 15

Add explicit “sanity gates” and fail fast with clear messages.

Examples: - If Stage 2 output < (min_expected) → raise `PipelineDataQualityError` and stop (or continue but mark as degraded and alert). - If Stage 3 success rate < X% → emit alert and still generate report with “partial enrichment” banner.

Add log rotation / retention.

Adopt rotating logs or retention cleanup. Python’s logging cookbook provides rotating patterns. 12

Priority of implementation

Short term. Run_id propagation + sanity gates + basic metrics (even as JSON) deliver immediate operational value. Prometheus/OpenTelemetry can follow.

Configuration and security

Problems found

A hard-coded API key is present in source code.

`src/etapas/etapa4.py::_obtener_utm` passes `apikey='3638141efe...'` directly in code (around lines 104–108). This is a critical secret-management issue: keys can leak via repository history, sharing, or logs.

Secrets stored in Excel config is fragile and difficult to manage professionally.

Stage 3 loads the Mercado Público API key from `PIVOT_MAESTRO.xlsx` via `Config.cargar_desde_pivot()` and looks for 'API Key' (e.g., `src/etapas/etapa3.py::_cargar_api_key`, around lines 43–52). Excel as a secret store: - lacks audit trails and access control typical of secret managers - is easy to accidentally share - is hard to rotate safely

Excel-driven configuration has implicit limits and schema assumptions.

`Config.cargar_desde_pivot` only reads rows 5–50 and uses fixed columns B and E (around lines 161–169). Configuration beyond row 50 is silently ignored, and there is no schema versioning or validation beyond “sheet exists.”

Configuration is not environment-separated.

The Twelve-Factor approach recommends storing deploy-varying config (including credentials) in environment variables, not embedded in code or committed config files. 16

Impact

High. Hard-coded credentials are a serious security and governance risk. Configuration fragility is a long-term operational risk.

Solution proposed

Immediate: remove hard-coded keys and rotate them.

- Move the UTM API key to an environment variable (or a secret store). - Implement a “fail closed” behavior if the key is missing (or fall back to defaults but trigger an alert).

Adopt a typed Settings model with environment-variable support.

Pydantic Settings provides structured parsing/validation for environment-based config. ¹⁷

This gives you: required fields, types, defaults, nested config, and consistent behavior across environments.

Example (illustrative):

```
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    mercado_publico_ticket: str
    cmf_api_key: str | None = None
    fx_provider_url: str = "https://api.exchangerate.host/latest"

    class Config:
        env_prefix = "LICIT_"
```

Treat Excel as an analyst-facing UI, not the runtime source of truth for secrets.

A professional compromise that preserves analyst workflows: - Keep filters in Excel (if needed), but export them to a validated, versioned format (YAML/JSON) during a “build step.” - Never store credentials in Excel; store them in env/secret manager. - Maintain a schema version and validate it during pipeline startup.

Add secret scanning and prevention.

OWASP guidance explicitly states secrets should not be hardcoded or stored in source code, and recommends detection/pre-commit scanning. ¹⁸

Priority of implementation

Quick win (for removing hardcoded key) → short term (for Settings refactor). Hard-coded secret removal is urgent; configuration professionalization is the next step.

Experience of use and documentation

Problems found

A new developer will struggle to run this in <30 minutes due to environment and dependency issues.

- Missing dependencies in `requirements.txt` (requests, pytz). - sys.path hacks indicate the project is not installable as a package. - Runtime expects external Excel files with implied schemas (not versioned, not validated beyond basic checks).

Execution UX is split across multiple entrypoints with inconsistent messaging.

There is: - `run_pipeline.py` CLI orchestrator - Stage-level `main()` functions - A Tkinter launcher (`lanzador_licitaciones.py`) that runs via subprocess and streams console output
These are useful, but a professional setup consolidates around a single CLI with consistent flags, exit codes, and help output.

User messages are log-heavy, but not always “actionable.”

Many logs are informational; fewer provide “what to do next” remediation (e.g., “Missing sheet 06-FILTROS → open pivot → validate header row 5 → re-run stage2”).

Technical documentation gaps.

README exists, but key “professional onboarding” pieces are missing or implied: - `.env.example` / configuration reference - architecture diagram and artifact contract - troubleshooting section by stage - “how to run tests” and “how to add a new stage” - sample test fixtures/data (sanitized)

Impact

Medium. The system appears usable for current maintainers, but onboarding and safe handoff are expensive.

Solution proposed

Create a single, professional CLI with subcommands.

Example: - `licitaciones run --stages 0,1,2,3,4,5` -
`licitaciones resume --run-id ...` - `licitaciones validate-config` - `licitaciones doctor` (checks environment, required files, permissions)

Add a developer “quickstart kit.”

- Pin dependencies (consider `pyproject.toml` + lock file strategy) - Provide `make setup`, `make test`, `make run-sample` - Provide a small sanitized sample Excel (or CSV/Parquet equivalent) for test runs that doesn’t include client-sensitive data.

Document artifact flow and stage contracts.

A short “Architecture” doc should state: - inputs/outputs per stage - expected columns and invariants - where manual analyst work is preserved and how

Priority of implementation

Short term. Fix installability first (dependencies + packaging). Then consolidate to a single CLI and improve docs.

Business value

Problems found

Current value is strong but mostly “batch automation,” not yet a strategic procurement intelligence asset.

The system already: - reduces 12k to ~200 - enriches and produces executive Excel - preserves manual analyst work via incremental logic

But the pipeline's outputs and decisions (filters, matches, enrichment success/failure reasons) are not yet captured as a durable, analyzable knowledge base.

Limited feedback loop from analysts to improve filtering quality systematically.

There is some incremental/suggestion tooling, but no explicit model of: - false positives / false negatives
- analyst edits explaining why a tender matters - learning over time

Opportunity to productize: data quality, trend insights, and proactive alerts.

Right now, if "something changes" (market shift, new procurement patterns), the system doesn't automatically surface trends or risks.

Impact

Medium. The automation is already impactful; strategic evolution can multiply value but requires solid foundations (config, reliability, data contracts).

Solution proposed

Highest-value functional expansions for IDOM Consulting (once foundations are stable)

- **Analyst feedback capture → continuous improvement loop:** record analyst actions (kept/removed/priority) and use that to refine keyword filters and/or train a classifier (even a simple logistic regression baseline can beat static keywords over time).
- **Procurement intelligence dashboard:** summarize by organism, region, category, budget bands, timeline, and trend vs last 7/30/90 days.
- **Proactive notifications:** alert when high-value tenders match strategic categories or when a tender is near closing and unreviewed.
- **Client-specific profiles:** maintain "client focus profiles" (keywords, organisms, budgets, regions) so reports are tailored per account rather than one global filter set.
- **Data lineage & auditability:** preserve "why it matched" (keyword hits, taxonomy match, enrichment fields used) to justify inclusion and speed analyst review.

Strategic evolution path - Treat this as a "procurement signals platform": - durable storage of tenders + enrichments + decisions - standardized API for consumers (Power BI, CRM, internal portals) - governance around config, secrets, and deployments

Priority of implementation

Long term (after reliability + architecture upgrades). Business-value additions compound best when core operations are stable and observable.

Roadmap of the 10 most impactful changes

1. **Remove the hard-coded CMF API key and rotate secrets** (`src/etapas/etapa4.py::_obtener_utm`, lines ~104-108).
Priority: **Quick win.**
Rationale: eliminates immediate credential leak risk; aligns with "no secrets in code." ¹⁹
2. **Fix dependency management and make installs reproducible** (`requirements.txt` missing `requests`, `pytz`; add version pinning / move to `pyproject.toml`).

Priority: **Quick win.**

Rationale: enables reliable onboarding and CI; aligns with modern packaging standards. ⁹

3. **Remove** `sys.path.append` **hacks by packaging the project properly** (multiple stage files).

Priority: **Short term.**

Rationale: reduces environment-specific failures; supports clean CLI entrypoints.

4. **Introduce a** `PipelineContext` **+ explicit artifacts between stages (stop “latest file glob” coupling)** (`run_pipeline.py`, `src/etapas/etapa3.py::_obtener_archivo_entrada`, `src/etapas/etapa4.py::_obtener_archivo`).

Priority: **Short term.**

Rationale: makes stages replaceable and prevents cross-run artifact confusion.

5. **Centralize HTTP behavior (session, retries, backoff, timeouts) used by Stage 0/3/4.**

Priority: **Short term.**

Rationale: Requests does not retry by default; central policy improves reliability and maintainability. ⁴

6. **Add Stage 3 checkpointing + durable caching (resume enrichment without losing progress)** (`src/etapas/etapa3.py::_enriquecer_licitaciones`).

Priority: **Short term.**

Rationale: biggest resilience improvement in the longest-running external dependency step.

7. **Refactor Stage 2 filtering to avoid repeated normalization and row-wise apply loops** (`src/etapas/etapa2.py`, `src/utils/text_processing.py::_contiene_palabras_clave`).

Priority: **Short term.**

Rationale: improves speed and predictability; reduces CPU scaling risk.

8. **Add sanity checks + anomaly alerts (0 filtered, low enrichment rate, missing columns)** (pipeline-wide).

Priority: **Short term.**

Rationale: prevents silent failures and accelerates detection of schema drift.

9. **Add basic operational metrics and structured logging with a shared run_id** (`src/utils/logger.py`, `orchestrator`).

Priority: **Short term.**

Rationale: enables production diagnosis and trend monitoring; prepares for Prometheus/OpenTelemetry adoption. ²⁰

10. **Create a minimal unit test suite for the most failure-prone logic (filters, API parsing, report invariants)** (new `tests/` with pytest fixtures).

Priority: **Short term → long term** (start small, grow steadily).

Rationale: prevents regressions when refactoring architecture; pytest fixtures support modular scalable tests. ²¹

- 2 8 21 **How to use fixtures — pytest documentation**
https://pytest.org/en/8.1.x/how-to/fixtures.html?utm_source=chatgpt.com
- 3 11 **How to automatically rerun your workflow when it fails - Prefect**
https://docs.prefect.io/v3/how-to-guides/workflows/retries?utm_source=chatgpt.com
- 4 **Requests Documentation**
https://app.readthedocs.org/projects/requests/downloads/pdf/v2.8.1/?utm_source=chatgpt.com
- 5 **stamina 25.2.0 documentation**
https://stamina.hynek.me/?utm_source=chatgpt.com
- 6 **The Twelve-Factor App**
https://www.12factor.net/?utm_source=chatgpt.com
- 7 **PEP 484 – Type Hints | peps.python.org**
https://peps.python.org/pep-0484/?utm_source=chatgpt.com
- 9 **pyproject.toml specification - Python Packaging User Guide**
https://packaging.python.org/specifications/declaring-project-metadata/?utm_source=chatgpt.com
- 12 **Logging Cookbook — Python 3.15.0a6 documentation**
https://docs.python.org/bn-in/dev/howto/logging-cookbook.html?utm_source=chatgpt.com
- 13 **Instrumenting | client_python**
https://prometheus.github.io/client_python/instrumenting/?utm_source=chatgpt.com
- 14 20 **The Twelve-Factor App**
https://www.12factor.net/logs?utm_source=chatgpt.com
- 15 **Writing client libraries | Prometheus**
https://prometheus.io/docs/instrumenting/writing_clientlibs/?utm_source=chatgpt.com
- 16 **The Twelve-Factor App**
https://www.12factor.net/config?utm_source=chatgpt.com
- 17 **Settings Management - Pydantic**
https://docs.pydantic.dev/2.6/concepts/pydantic_settings/?utm_source=chatgpt.com
- 18 19 **OWASP DevSecOps Guideline - v-0.2 | OWASP Foundation**
https://owasp.org/www-project-devsecops-guideline/latest/01a-Secrets-Management?utm_source=chatgpt.com